

Assignments and Arithmetic Operators

ELEC1006: ENGINEERING COMPUTING

- Assignment (=)
- Arithmetic operators (+, -, *, /, %)
- Compound assignment (+=, -=, *=, /=, %=, >>=, <<=, &=, ^=, |=)
- Increase and decrease (++ , --)
- Relational and equality operators (<, >, <=, >=, ==, !=)
- Logical operators (!, &&, ||)
- Bitwise operators (&, |, ^, ~, <<, >>)
- Conditional operator (?)
- Comma operator (,)
- Bitwise operator
- Explicit type casting operator
- Precedence of operators

- Assignment operator assigns a **value** to a **variable**

a=5;

- Most important rule in 'assignment' is '**right-to-left**', i.e., the operation always takes place from the right to left. **a=b;** (whatever in **a** will be overwritten by the contents of **b**)
- **a=2+(b=5);**  **b=5; a=2+b;**
- **a=b=c=5;**  **c=5; b=c; a=b;**

- The **=** can be used to assign a value to multiple variables:

$$x = y = z = 5;$$

- Value of = is the value that is assigned
- Associates right to left:

$$x = (y = (z = 5)) ;$$

- Look at the following statement:

New value of sum	Old value of sum
	
sum	sum
=	+
	1;

`sum + 1` adds 1 to the variable `sum` and stores the result back into variable `sum` using the assignment `=`.

Assignments & Arithmetic Operators

Statement	What It Does	Value of x After the Statement
<code>x = x + 4;</code>	Adds 4 to x	10
<code>x = x - 3;</code>	Subtracts 3 from x	3
<code>x = x * 10;</code>	Multiplies x by 10	60
<code>x = x / 2;</code>	Divides x by 2	3
<code>x = x % 4</code>	Makes x the remainder of x / 4	2

Arithmetic Operators

(+, -, *, /, %)

- % is the modulo operation (**not a percentage calculation**) – it calculates the remainder of the division between the two values:

`a=11 % 3;`



`a = 2;`

Algebraic Expressions

Algebraic Expression	Operation	C++ Equivalent
$6B$	6 times B	<code>6 * B</code>
$(3)(12)$	3 times 12	<code>3 * 12</code>
$4xy$	4 times x times y	<code>4 * x * y</code>

Algebraic Expressions

- Multiplication requires an operator:
 $Area = lw$ is written as $Area = l * w;$
- There is no exponentiation operator:
 $Area = s^2$ is written as $Area = pow(s, 2);$
- Parentheses may be needed to maintain order of operations:

$$m = \frac{y2 - y1}{x2 - x1}$$

is written as

$$m = (y2 - y1) / (x2 - x1);$$

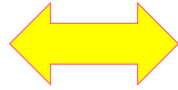
NOT

$$m = y2 - \frac{y1}{x2} - x1$$

$$m = y2 - y1 / x2 - x1;$$

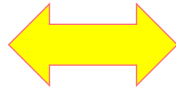
Compound Assignment

`a += 5;`



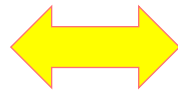
`a = a + 5;`

`a -= 5;`



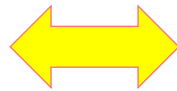
`a = a - 5;`

`a /= b;`



`a = a / b;`

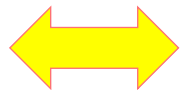
`a *= b+1;`



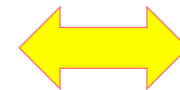
`a = a * (b+1);`

Increase and decrease

`a ++;`



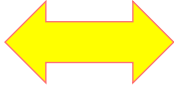
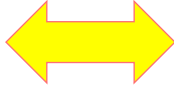
`a += 1;`



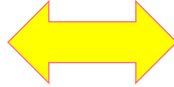
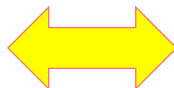
`a = a + 1;`

Compound Assignment

Increase and decrease

<code>a ++;</code>		<code>a += 1;</code>		<code>a = a + 1;</code>
<code>++a ;</code>				<code>a = a + 1;</code>

If a=2, b=5

<code>c=a ++*b;</code>		<code>c=a*b;</code> <code>a=a++;</code>	<code>c=10;</code> <code>a=3;</code>
<code>c=++a *b;</code>		<code>a=a++;</code> <code>c=a*b;</code>	<code>a=3;</code> <code>c=15;</code>

Same rules apply to --

More info on C++

- [1] cplusplus.com: Operators
<https://cplusplus.com/doc/tutorial/operators/>

Other Common Operators

ELEC1006: ENGINEERING COMPUTING

Operators

- Assignment (=)
- Arithmetic operators (+, -, *, /, %)
- Compound assignment (+=, -=, *=, /=, %=, >>=, <<=, &=, ^=, |=)
- Increase and decrease (++ , --)
- Relational and equality operators (<, >, <=, >=, ==, !=)
- Logical operators (!, &&, ||)
- Bitwise operators (&, |, ^, ~, <<, >>)
- Conditional operator (?)
- Comma operator (,)
- Bitwise operator
- Explicit type casting operator
- Precedence of operators.

Used to evaluate a comparison between two expressions

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

== is not the
same as =

e.g. a=2, b=3, c=6;

```
(a == 5)      // evaluates to false since a is not equal to 5.  
(a*b >= c)    // evaluates to true since (2*3 >= 6) is true.  
(b+4 > a*c)   // evaluates to false since (3+4 > 2*6) is false.  
((b=2) == a)  // evaluates to true.
```

What will this statement produce **b=2==a** ????

Logical Operators

! Is the logic **NOT** operation (only has ONE operand)

&& is the logic **AND** operation (needs TWO operand)

|| is the logic **OR** operation (Two operand)

eg. `a=3, b=5;`

`!(a==b)` is true

(similar to `a!=b`)

`(a>3)&&(b<7)` is false

`(a>3)|| (b<7)` is true

Conditional Operator

- Evaluates an expression returning a value if that expression is true and a different one if the expression is false.

- Its format is:

condition ? result1 : result2

True – result 1

False – result 2

```
7==5 ? 4 : 3
7==5+2 ? 4 : 3
5>3 ? a : b
a>b ? a : b
```

```
#include <iostream>
using namespace std;

int main ()
{
    int a,b,c;

    a=2;
    b=7;
    c = (a>b) ? a : b;

    cout << c;

    return 0;
}
```


Comma Operator

- Used to separate two or more expressions that are included where only one expression is expected
- When the set of expressions has to be evaluated for a value, only the rightmost expression is considered:

`a = (b = 2, b+3);`  `b=2; a=b+3;`

Bitwise Operator

- Single bit operations.
- 1 bit only has binary (2) states.
 - 0 (FALSE) or 1 (TRUE).
- NOT inverts states, i.e. $1 \rightarrow 0$ and $0 \rightarrow 1$.
- Refer to the Truth Table for the possible states.
- SHL refers to shift left operation:
 - $001 \rightarrow 010$.
- SHR refers to shift right operation:
 - $110 \rightarrow 011$.

operator	asm equivalent
&	AND
	OR
^	XOR
~	NOT
<<	SHL
>>	SHR

Truth Table

Input 1	Input 2	AND	OR	XOR
0	0	0	1	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Type Casting Operator

- Operations are performed between operands of the same type.
- If not of the same type, C++ will convert one to be the type of the other
- This can impact the results of calculations.

e.g: float a=10.0;

int b=4;

int c=a/b;



c = 2 instead of c = 2.5 due to c being an integer instead of float.

Precedence of Operators

Level	Precedence group	Operator	Description	Grouping
1	Scope	::	scope qualifier	Left-to-right
2	Postfix (unary)	++ --	postfix increment / decrement	Left-to-right
		()	functional forms	
		[]	subscript	
		. ->	member access	
3	Prefix (unary)	++ --	prefix increment / decrement	Right-to-left
		~ !	bitwise NOT / logical NOT	
		+ -	unary prefix	
		& *	reference / dereference	
		new delete	allocation / deallocation	
		sizeof	parameter pack	
		(type)	C-style type-casting	
4	Pointer-to-member	.* ->*	access pointer	Left-to-right
5	Arithmetic: scaling	* / %	multiply, divide, modulo	Left-to-right
6	Arithmetic: addition	+ -	addition, subtraction	Left-to-right
7	Bitwise shift	<< >>	shift left, shift right	Left-to-right
8	Relational	< > <= >=	comparison operators	Left-to-right
9	Equality	== !=	equality / inequality	Left-to-right
10	And	&	bitwise AND	Left-to-right
11	Exclusive or	^	bitwise XOR	Left-to-right
12	Inclusive or		bitwise OR	Left-to-right
13	Conjunction	&&	logical AND	Left-to-right
14	Disjunction		logical OR	Left-to-right
15	Assignment-level expressions	= *= /= %*= += -=>>= <<= &= ^= =	assignment / compound assignment	Right-to-left
		?:	conditional operator	
16	Sequencing	,	comma separator	Left-to-right

More info on C++

- [1] cplusplus.com: Operators
<https://cplusplus.com/doc/tutorial/operators/>

if Conditional Branch

ELEC1006: ENGINEERING COMPUTING

Conditional Branches

- Defined as a condition that causes the program to change the path of execution based on the value of an expression.
- Forms of conditional branches:
 - *if* statement. ←
 - *switch* statement.

if Statement

- The '*if*' (a keyword) is to execute a statement or a block of statements only if a condition is satisfied. The form is:
 - *if* (condition) statement
 - e.g. *if* (x>90)
 cout << "A very good score!" << endl;

This 'cout statement' will only be executed if the value in x is greater than 90.

Multiple Lines of Code

- if multiple lines of code is to be executed under an *if* statement, {} should be used.
- Do **NOT** put ; after the *if (condition)* statement.
- e.g. **if** (x>90)

{

cout << "Congratulation!"<< endl;

cout << "A very good score of ";

cout << x << "." << endl;

}

Two Conditional Branches

- The **else** statement can be used to specify what to do if the 'if' condition is not met.
- e.g. **if** (x>90) **// Condition 1**
 cout << "Congratulation!"<<endl;
else **// Condition 2**
 cout << "You can improve your score";

Multiple Conditional Branches

- Use **if – else if – else** for a range of conditional branches.

– e.g.

```
if (x<0)          // Condition 1
    cout << "x is negative." << endl;
else if (x>0)     // Condition 2
    cout << "x is positive." << endl;
else              // Condition 3
    cout << "x is 0." << endl;
```

Commas in *if* Statement

- Use of commas ,
- e.g.

```
int a=1,b=2,c=10,d=20;
```

```
if (a<0) c*=10, d*=20;
```

```
else if (a<b) c/=10, d/=20;
```

```
else c=b;
```

Previous code is equivalent to:

```
int a=1;
int b=2;
int c=10;
int d=20;
if (a<0)
{
    c=c*10;
    d=d*20;
}
else if (a<b)
{
    c=c/10;
    d=d/20;
}
else
    c=b;
```

— Too long?

More info

- [1] cplusplus.com: Statements and flow control
<https://cplusplus.com/doc/tutorial/control/>
- [2] learncpp.com: 5.2 – If statements
<https://www.learncpp.com/cpp-tutorial/52-if-statements/>

switch Conditional Branch

ELEC1006: ENGINEERING COMPUTING

Conditional Branches

- Defined as a condition that causes the program to change the path of execution based on the value of an expression.
- Forms of conditional branches:
 - *if* statement.
 - *switch* statement. ←

switch Statement

- `switch` statement is a natural choice for menu-driven program:
 - display the menu
 - then, get the user's menu selection
 - use user input as `expression` in `switch` statement
 - use menu choices as `expr` in `case` statements
- `if-else` can be used in place of `switch`.

switch Statement Format

```
switch (expression) //integer
{
    case exp1: statement1;
    case exp2: statement2;
    ...
    case expn: statementn;
    default:    statementn+1;
}
```

break Statement

- Used to exit a `switch` statement
- If it is left out, the program "falls through" the remaining statements in the `switch` statement
- Example:

```
switch (n) {  
    case 'a':  
        cout << "a has been given.\n";  
        break;  
    case 'b':  
        cout << "b has been given.\n";  
        break;  
    default:  
        cout << "unknown character!\n";  
        break;  
}
```

Alternative Program

- Use `if - else if - else` statement as an alternative to `switch`:
- Example:

```
if (n == 'a' )  
    cout << "a has been given.\n";  
else if (n == 'b' )  
    cout << "b has been given.\n";  
else  
    cout << "unknown character!\n";  
}
```

More info

- [1] cplusplus.com: Statements and flow control
<https://cplusplus.com/doc/tutorial/control/>
- [2] learncpp.com: 5.3 – Switch statements
<https://www.learncpp.com/cpp-tutorial/53-switch-statements/>

while Iteration Statement

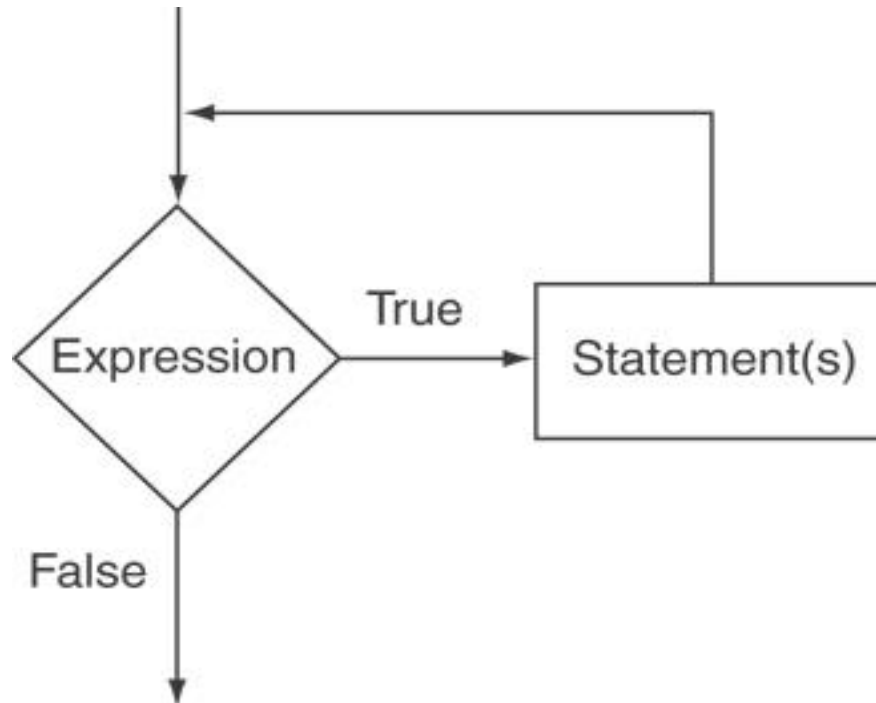
ELEC1006: ENGINEERING COMPUTING

Iteration Statement (loops)

- Loops are used to repeat a statement a certain number of times when a condition is met.
- There are three types of loops:
 - *while* loop. ←
 - *do-while* loop.
 - *for* loop.

The *while* Loop

- The format is:
while (expression or condition) { Statement(s) }
– repeat while “expression or condition” is **true**.
- The logical flow of while loop:

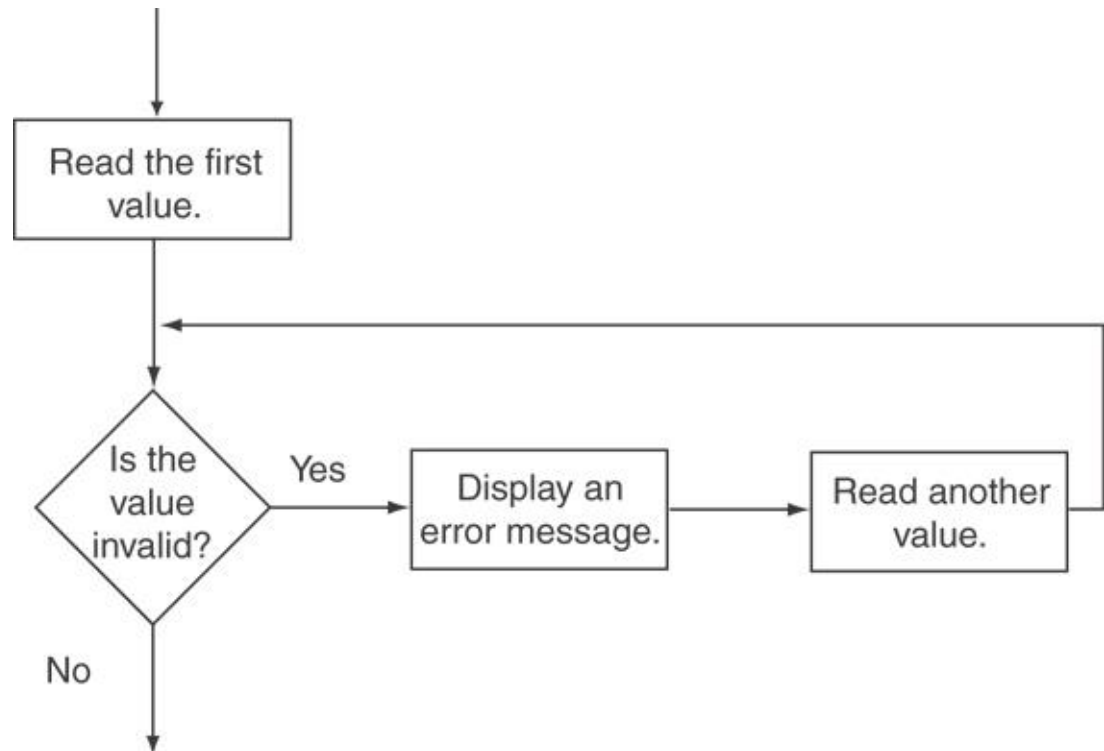


Example

```
// custom countdown using while
#include <iostream>
using namespace std;
int main ()
{
    int n;
    cout << "Enter the starting number: ";
    cin >> n;
    while (n>0) // Don't put ; at the end
    {
        cout << n << ", ";
        --n; // Decrement n by 1 in every loop
    }
    cout << "FIRE!\n";
    return 0;
}
```

Input validation

```
cout << "Enter a number less than 10: ";  
cin >> number;  
while (number >= 10)  
{  
    cout << "Invalid Entry!"  
        << "Enter a number less than 10: ";  
    cin >> number;  
}
```



Tips: When to use *while*

- Use `while` statement when the number of times the loop is required to run is unknown.
- Use `while (1)` to form an infinite loop especially when using a simple program that runs for long periods of time, e.g. mobile phone call:

`while (1)`

```
{  
    read_input();      // 1) Get network data  
    process_data();    // 2) Convert into sound signal  
    output_data();     // 3) Play signal on speaker  
    if call_end()  
        break;        // Break the while loop if call ends  
}
```

More info

- [1] cplusplus.com: Statements and flow control
<https://cplusplus.com/doc/tutorial/control/>
- [2] learncpp.com: 5.5 – While statements
<https://www.learncpp.com/cpp-tutorial/55-while-statements/>

do-while Iteration Statement

ELEC1006: ENGINEERING COMPUTING

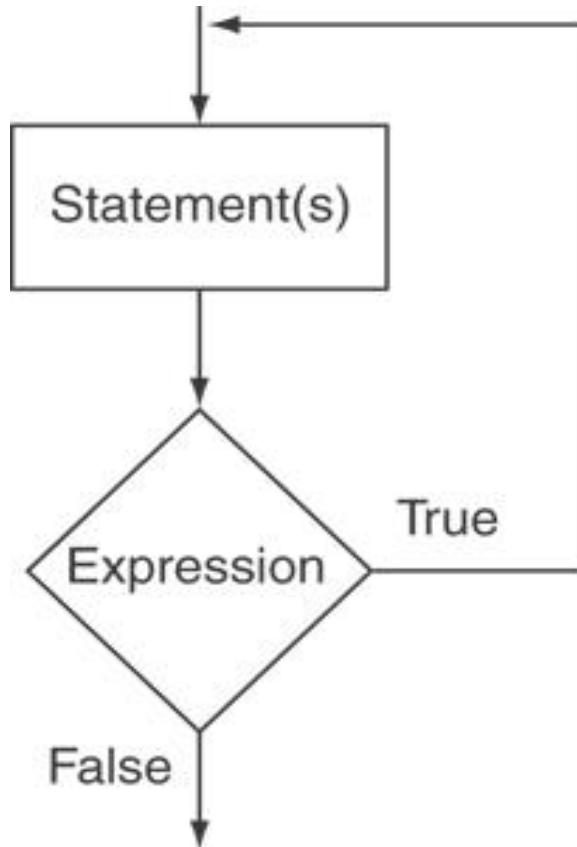
Iteration Statement (loops)

- Loops are used to repeat a statement a certain number of times when a condition is met.
- There are three types of loops:
 - *while* loop.
 - *do-while* loop. ←
 - *for* loop.

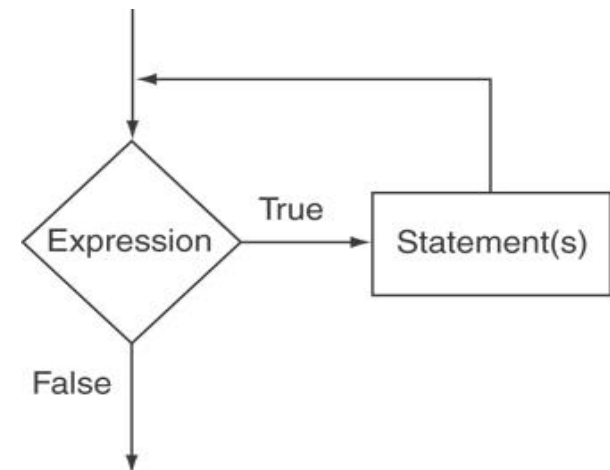
The *do-while* Loop

- The format is:
do { statements } *while* (expression or condition);
- Only difference with the *while* statement is that *do-while* evaluates the condition **after executing** the statements.

do-while Loop Logic



while Loop Logic



Example

```
// Number echo
#include <iostream>
using namespace std;
int main ()
{
    unsigned long n;
    do {
        cout << "Enter number (0 to end): ";
        cin >> n;
        cout << "You entered: " << n << "\n";
    } while (n != 0); // Must use ;

    return 0;
}
```


Notes on *do-while*

- Loop always executes at least once.
- Execution continues as long as the *expression* is true and stops the repetition when the *expression* becomes false.
- Useful in menu-driven programs to bring user back to menu to make another choice.

Menu –driven Program

```
#include <iostream>

using namespace std;

int main()
{
    int selection;           // Selection variable

    do {
        cout << "Please make a selection: \n";           // List menu
        cout << "1) Addition\n";
        cout << "2) Subtraction\n";
        cout << "3) Multiplication\n";
        cout << "4) Division\n";
        cin >> selection;           // Get user menu selection
    } while (selection != 1 && selection != 2 && selection != 3 && selection != 4);

    // Do something with the selection here

    return 0;
}
```

More info

- [1] cplusplus.com: Statements and flow control
<https://cplusplus.com/doc/tutorial/control/>
- [2] learncpp.com: 5.6 – Do while statements
<https://www.learncpp.com/cpp-tutorial/56-do-while-statements/>



for Iteration Statement

ELEC1006: ENGINEERING COMPUTING

Iteration Statement (loops)

- Loops are used to repeat a statement a certain number of times when a condition is met.
- There are three types of loops:
 - *while* loop.
 - *do-while* loop.
 - *for* loop. ←

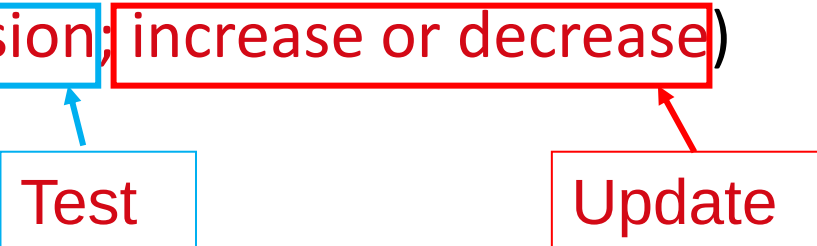
The *for* Loop

- *for* loop is used when we know how many iterations (loops) are required.
- Test each expression before each iteration in the *for* loop and so, it is known as a pre-test loop similar to *while* loop.
 - Note: *do-while* loop is known as a post-test loop, where the statements executed at least once (one iteration) before the expression is checked.

for Loop Syntax

- The format is:

```
for (initialisation; expression; increase or decrease)
{
    statements
}
```



– Note: No ; after the)

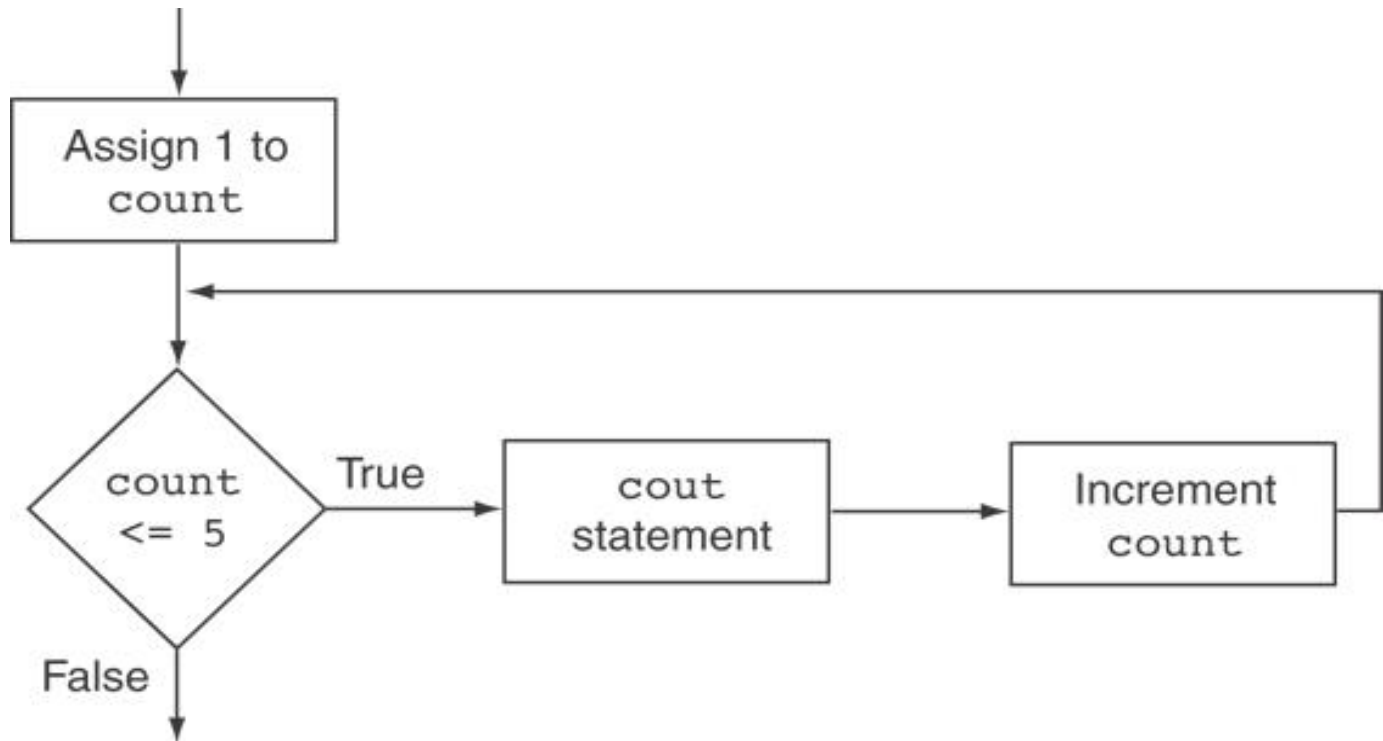
The mechanics:

- Perform initialisation
- Evaluate test expression
 - If true, execute statement
 - If false, terminate loop execution
- Execute update, then re-evaluate test expression

Example

```
for (int count = 1; count <= 5; count++)  
{  
    cout << "Hello" << endl;  
}
```

Flowchart



while Loop Equivalent

```
for (int count = 1; count <= 5; count++)  
{  
    cout << "Hello" << endl;  
}
```

for loop is like an advanced *while* loop

```
int count=1;  
while (count <= 5)  
{  
    cout << "Hello" << endl;  
    count++;  
}
```

Variations using commas

- Can also have multiple **initialisation** and **update** expressions. Separate the statements with **commas** , :

```
int x, y;
for (x=1, y=1; x <= 5; x++, y++)
{
    cout << x << " plus " << y
        << " equals " << (x+y)
        << endl;
}
```

Initialisations

updates

Scope of Variables

- You can declare variables in the **initialisation** expression.
- Variable `i` only exist in the `for` loop `{ ... }`, not outside of it.

```
int sum = 0;
for (int i = 0; i <= 10; i++)
{
    sum += i;           // sum = sum + i
}
```

More info

- [1] cplusplus.com: Statements and flow control
<https://cplusplus.com/doc/tutorial/control/>
- [2] learncpp.com: 5.7 – For statements
<https://www.learncpp.com/cpp-tutorial/57-for-statements/>

Nested Loops & Exit Statement

ELEC1006: ENGINEERING COMPUTING

Combining Loops

- Loops can be combined together (nested loops).
 - The statement within a loop can contain another loop:

```
int i, j;  
for (i = 0; i < 4; i++)  
{  
    for (j = 0; j < 4; j++)  
    {  
        // statements  
    }  
}
```

Nested Loops

- Inner loop goes through all repetitions for each repetition of outer loop
- Inner loop repetitions complete sooner than outer loop
- Total number of repetitions for inner loop is the product of number of repetitions of the two loops.

exit() Statement

- Use `exit(n)` to exit the program.
 - `n` is a specific exit code:
 - 0 for successful exit.
 - 1 or non-zero number for failure as mentioned below.
- **exit(1)**: It indicates abnormal termination of a program perhaps as a result a minor problem in the code.
- **exit(2)**: It is similar to `exit(1)` but is displayed when the error occurred is a major one. This statement is rarely seen.
- **exit(127)**: It indicates command not found.
- **exit(132)**: It indicates that a program was aborted (received **SIGILL**), perhaps as a result of illegal instruction or that the binary is probably corrupt.
- **exit(133)**: It indicates that a program was aborted (received **SIGTRAP**), perhaps as a result of dividing an integer by zero.
- **exit(134)**: It indicates that a program was aborted (received **SIGABRT**), perhaps as a result of a failed assertion.
- **exit(136)**: It indicates that a program was aborted (received **SIGFPE**), perhaps as a result of floating point exception or integer overflow.
- **exit(137)**: It indicates that a program took up too much memory.
- **exit(138)**: It indicates that a program was aborted (received **SIGBUS**), perhaps as a result of unaligned memory access.
- **exit(139)**: It indicates **Segmentation Fault** which means that the program was trying to access a memory location not allocated to it. This mostly occurs while using pointers or trying to access an out-of-bounds array index.
- **exit(158/152)**: It indicates that a program was aborted (received **SIGXCPU**), perhaps as a result of CPU time limit exceeded.
- **exit(159/153)**: It indicates that a program was aborted (received **SIGXFSZ**), perhaps as a result of File size limit exceeded.

Example

```
#include <iostream>
using namespace std;

int main ()
{
    unsigned long n=1;
    for (n; n<5; n++)
    {
        cout << "hello world\n";
        if(n==2)
            exit(1);
    }

    cout << "this is the end of the program.";

    return 0;
}
```

More info

- [1] cplusplus.com: Statements and flow control
<https://cplusplus.com/doc/tutorial/control/>
- [2] cplusplus.com: function exit
<https://cplusplus.com/reference/cstdlib/exit/?kw=exit>
- [3] learncpp.com: 5.7 – For statements
<https://www.learncpp.com/cpp-tutorial/57-for-statements/>
- [4] Exit codes in C/C++ with Examples
<https://www.geeksforgeeks.org/exit-codes-in-c-c-with-examples/>